

Lista de exercícios 11 de C24 - Inteligência Artificial

Redes neurais recorrentes

Exercícios teóricos

1. Imagine que você está desenvolvendo um sistema para prever o próximo caractere em um texto. Por que uma Rede Neural Multicamadas (MLP) tradicional teria dificuldades nessa tarefa em comparação a uma RNN?

- A) Porque MLPs exigem que a entrada e a saída tenham sempre o mesmo tamanho fixo.
- B) Porque MLPs tratam cada caractere de forma independente, ignorando o contexto e a ordem das letras anteriores.
- C) Porque MLPs não suportam o uso de funções de ativação não lineares em suas camadas ocultas.
- D) Porque MLPs são otimizadas apenas para dados espaciais, como imagens, e não para vetores numéricos.

2. Durante o treinamento de uma RNN, um engenheiro nota que o erro da rede parou de diminuir e os gradientes se tornaram quase nulos para camadas iniciais de sequências longas. Qual problema está ocorrendo e qual é a consequência para o modelo?

- A) Explosão do gradiente; a rede está fazendo atualizações de peso excessivamente grandes.
- B) Sobreajuste (Overfitting); a rede memorizou os dados de treino perfeitamente.
- C) Desaparecimento do gradiente; a rede não consegue aprender dependências de longo prazo.
- D) Instabilidade numérica; o modelo está sofrendo oscilações bruscas no erro.

3. Na arquitetura de uma RNN tradicional, os pesos das conexões recorrentes (W_h) são compartilhados em todos os passos de tempo t . Qual é a principal justificativa para esse compartilhamento?

- A) Permitir que a rede aprenda uma função de transição única que seja aplicada de forma consistente a qualquer parte da sequência.
- B) Reduzir o tempo de processamento ao eliminar a necessidade de realizar multiplicações de matrizes.
- C) Garantir que a rede sempre produza uma saída de tamanho fixo, independentemente da entrada.
- D) Evitar que a rede precise utilizar funções de ativação como a tangente hiperbólica ($\tanh$).

4. Um modelo de Deep Learning é treinado para analisar uma mensagem de rede social e classificá-la como "Positiva", "Negativa" ou "Neutra". Qual tipo de arquitetura RNN descreve corretamente esse fluxo de dados?

- A) Um-para-Vários (One-to-Many).
- B) Vários-para-Um (Many-to-One).
- C) Vários-para-Vários Síncrono (Many-to-Many Synchronous).
- D) Um-para-Um (One-to-One).

5. Por que a função de ativação ReLU, embora popular em redes convolucionais, é geralmente evitada na camada recorrente de RNNs simples, sendo preferidas as funções tanh ou Sigmoid?

- A) Porque a ReLU é muito lenta para calcular em processamentos sequenciais.
- B) Porque a ReLU pode levar à explosão do gradiente, já que suas saídas não são saturadas e podem crescer indefinidamente na recorrência.
- C) Porque a ReLU impede que a rede aprenda padrões negativos nos dados sequenciais.
- D) Porque a ReLU causa o desaparecimento imediato de toda a memória da rede após dois passos.

6. O treinamento via Backpropagation Through Time (BPTT) torna-se computacionalmente caro em sequências longas. O BPTT Truncado é usado como alternativa. Qual é a premissa lógica que justifica o uso dessa versão truncada?

- A) A premissa de que o estado oculto $h[t]$ já contém um resumo compacto das informações relevantes do passado.
- B) A premissa de que redes neurais só precisam de dois passos de tempo para aprender qualquer idioma.
- C) A premissa de que as entradas passadas não influenciam as saídas futuras em séries temporais.
- D) A premissa de que o erro de treinamento deve ser calculado apenas no primeiro passo da sequência.

7. Em uma célula LSTM, qual componente é responsável por decidir quais informações do estado da célula anterior (c_{t-1}) devem ser descartadas para dar lugar a novos dados?

- A) Porta de Entrada (Input Gate).
- B) Porta de Saída (Output Gate).
- C) Camada de Ativação tanh.
- D) Porta de Esquecimento (Forget Gate).

8. As Unidades Recorrentes com Portas (GRU) são consideradas versões simplificadas das LSTMs. Qual alteração estrutural fundamental permite que as GRUs sejam mais rápidas e tenham menos parâmetros?

- A) Elas fundem as portas de entrada e de esquecimento em uma única "porta de atualização" e não utilizam um estado de célula separado.
- B) Elas eliminam completamente o uso de funções de ativação Sigmoide.
- C) Elas processam a sequência de trás para frente, eliminando a dependência temporal.
- D) Elas substituem todas as matrizes de pesos por valores constantes pré-definidos.

9. Por que as LSTMs conseguem manter o fluxo do gradiente por muito mais tempo do que as RNNs tradicionais, evitando o desaparecimento do gradiente?

- A) Porque elas não utilizam o algoritmo de retropropagação (Backpropagation).
- B) Porque a atualização do estado da célula é baseada em operações lineares, criando um "atalho" para o gradiente.
- C) Porque elas limitam o tamanho das sequências de entrada a no máximo 10 palavras.
- D) Porque elas utilizam inicialização de pesos aleatória em cada iteração.

10. Considere um sistema de tradução automática (Vários-para-Vários Assíncrono). Por que o modelo espera ler a frase de entrada inteira antes de começar a produzir a tradução?

- A) Para economizar memória RAM durante o processo de inferência.
- B) Porque as RNNs assíncronas são incapazes de realizar cálculos em tempo real.
- C) Porque a ordem e o sentido das palavras podem mudar radicalmente entre os idiomas, exigindo o contexto global da frase.
- D) Para garantir que o número de palavras na tradução seja exatamente igual ao número de palavras na entrada.

11. Na estrutura de uma célula LSTM, qual é a função específica da "Porta de Entrada" (Input Gate)?

- A) Determinar qual proporção da nova informação candidata será armazenada no estado da célula.
- B) Decidir qual parte do estado da célula anterior deve ser removida.
- C) Selecionar qual informação do estado da célula será enviada para a saída final.
- D) Multiplicar a entrada bruta por um valor constante para normalizar os dados.

12. Qual é o objetivo principal do compartilhamento de pesos (Weight Sharing) em uma RNN ao longo dos passos de tempo?

- A) Reduzir o número de parâmetros do modelo e permitir o processamento de sequências de tamanhos variados.
- B) Impedir que a rede utilize funções de ativação não lineares.
- C) Garantir que a rede esqueça informações irrelevantes do início da sequência.
- D) Aumentar a complexidade da rede para que ela decore todos os dados de treino.

13. Qual arquitetura de RNN é a mais adequada para um sistema de "Análise de Sentimento", onde uma frase inteira é lida para gerar uma única classificação (e.g., Positivo ou Negativo)?

- A) Um-para-Vários (One-to-Many).
- B) Vários-para-Um (Many-to-One).
- C) Vários-para-Vários (Many-to-Many) síncrono.
- D) Um-para-Um (One-to-One).

14. Em uma arquitetura de Rede Neural Recorrente (RNN), o que representa o "estado oculto" (h_t)?

- A) É a função de custo que mede o erro entre a previsão e o valor real.
- B) É a camada de entrada que recebe os dados brutos da sequência.
- C) É o algoritmo de otimização usado para atualizar os pesos da rede.
- D) É um vetor que atua como uma memória, armazenando informações relevantes dos passos de tempo anteriores.

Exercício prático

Analizando Sentimentos com LSTM

Objetivo: Construir e treinar uma rede LSTM para classificar críticas de filmes do site IMDB como positivas (1) ou negativas (0).

Passo 1: Preparando o Ambiente e os Dados

Como textos não são números, o Keras já nos fornece o dataset onde as palavras foram substituídas por índices (números).

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Parâmetros
max_palavras = 10000 # Usaremos apenas as 10.000 palavras mais frequentes
max_len = 200       # Cortaremos cada crítica após 200 palavras

print("Carregando dados...")
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=max_palavras)

# Padronização: Críticas curtas ganham zeros, críticas longas são cortadas
X_train = sequence.pad_sequences(X_train, maxlen=max_len)
X_test = sequence.pad_sequences(X_test, maxlen=max_len)

print(f"Formato dos dados de treino: {X_train.shape}")
```

Passo 2: Construindo a Arquitetura

Aqui usamos uma camada de Embedding, que transforma números em vetores de significado, e a nossa LSTM.

```
model = Sequential([
    InputLayer(input_shape=(max_len,)),

    # Transforma índices de palavras em vetores densos de tamanho 128
    Embedding(max_palavras, 128),

    # A camada LSTM com 64 unidades (memória da rede)
    # dropout ajuda a evitar que a rede decore o dataset (overfitting)
    LSTM(64, dropout=0.2, recurrent_dropout=0.2),

    # Camada de saída para classificação binária
    Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])

model.summary()
```

Passo 3: Treinamento e Avaliação

Observarem a acurácia subindo a cada época.

```
print("Iniciando treinamento...")
# 2 ou 3 épocas já são suficientes para ver resultados em sala de aula
history = model.fit(X_train, y_train,
                   batch_size=64,
                   epochs=3,
                   validation_data=(X_test, y_test))

scores = model.evaluate(X_test, y_test, verbose=0)
print(f"Acurácia no Teste: {scores[1]*100:.2f}%")
```

Passo 4: O Desafio (Mão na Massa)

Tentem melhorar o modelo ou entender suas limitações. Realize as seguintes tarefas:

1. Mudança de Arquitetura: O que acontece com a acurácia e o tempo de treino se você dobrar o número de unidades da LSTM para 128?
2. Otimização: Tente usar uma camada Bidirectional(LSTM(64)). Isso permite à rede ler a frase de frente para trás e de trás para frente.

3. Teste Cego: Procurem uma crítica de filme curta (em inglês) na internet e tentem passar para o modelo prever.
- Dica: O código abaixo converte as palavras de uma frase qualquer nos mesmos índices usados no dataset. O código mapeia as palavras da frase digitada para os mesmos números (índices) que o dataset IMDB usou.

```
import numpy as np
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence

# 1. Carregar o dicionário de palavras do IMDB
word_index = imdb.get_word_index()

def classificar_frase(frase_texto, modelo, max_len=200):
    # Tratamento básico: letras minúsculas e separar palavras
    palavras = frase_texto.lower().split()

    # Converter palavras em índices (seguindo o padrão do Keras IMDB)
    # O Keras reserva os índices 0, 1 e 2 para [PAD], [START] e [UNK]
    indices = [1] # Começamos com o índice de "início de sequência"
    for palavra in palavras:
        # Se a palavra existe no dicionário e está dentro das 10k mais comuns
        indice = word_index.get(palavra, 2) # 2 é o índice para palavras desconhecidas (UNK)
        if indice < 10000:
            indices.append(indice + 3) # O shift de +3 é necessário no IMDB do Keras
        else:
            indices.append(2)

    # Garantir que a lista tenha o tamanho correto (Padding)
    indices_pad = sequence.pad_sequences([indices], maxlen=max_len)

    # Fazer a predição
    predicacao = modelo.predict(indices_pad, verbose=0)[0][0]

    sentimento = "POSITIVA" if predicacao > 0.5 else "NEGATIVA"
    confianca = predicacao if predicacao > 0.5 else 1 - predicacao

    print(f"\nFrase: '{frase_texto}'")
    print(f"Resultado: {sentimento} ({confianca*100:.2f}% de confiança)")

# --- EXEMPLO DE USO ---
# A frase deve ser em INGLÊS, já que o dataset original é inglês.
minha_critica = "this movie was amazing and the acting was great"
classificar_frase(minha_critica, modelo)

minha_critica_ruim = "it was a waste of time and very boring"
classificar_frase(minha_critica_ruim, modelo)
```